

# Vehicle Service Management System

*Project documentation: how it works and how the CRUDs are connected*

Prepared for Diar Mustafa - 2026-06-02

## 1. Executive summary

This document explains the uploaded Vehicle Service Management System project. The system is a full stack web application with a React + Vite frontend, an Express.js backend, and a MySQL database. The project manages users, roles, customers, vehicles, appointments, service records, parts, invoices, service reminders, public requests, mechanic applications, emergency pickups, and roadside subscriptions.

The backend exposes REST API endpoints. The frontend pages call these endpoints through a centralized Axios instance. Most admin/staff pages follow the same CRUD pattern: load data with GET, submit forms with POST, edit existing rows with PUT, and remove rows with DELETE. Authentication is done with JWT tokens stored in localStorage and checked by backend middleware and frontend protected routes.

## 2. Technology stack and folder structure

| Layer          | Main files/folders                                                                                                    | Purpose                                                                                 |
|----------------|-----------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Frontend       | Frontend/src, Frontend/src/pages, Frontend/src/components, Frontend/src/api/api.js                                    | React interface, public pages, dashboard pages, API calls, routing and protected pages. |
| Backend        | Backend/server.js, Backend/routes, Backend/controllers, Backend/config/db.js, Backend/middleware                      | Express server, API routes, database queries, JWT protection, MySQL connection.         |
| Database       | MySQL / XAMPP database: vehicle-service-management-system                                                             | Stores users, customers, vehicles, service data, requests, roles, and tokens.           |
| Authentication | Backend/controllers/auth.controller.js, Backend/middleware/auth.middleware.js, Frontend/src/routes/ProtectedRoute.jsx | Login, JWT creation, frontend route blocking, backend API protection.                   |
| UI assets      | Frontend/public, Frontend/src/assets, Frontend/src/index.css                                                          | Images, icons, CSS styling, public landing page visuals.                                |

## 3. High level architecture

User Browser

- > React frontend pages and components
- > Axios API object in Frontend/src/api/api.js
- > Express routes in Backend/routes/\*.routes.js
- > Controller functions in Backend/controllers/\*.controller.js
- > MySQL connection in Backend/config/db.js
- > MySQL database tables

The important idea is that the frontend never talks directly to MySQL. It only sends HTTP requests to the backend. The backend receives the request, validates token where required, executes SQL using mysql2, and returns JSON responses to the frontend.

## 4. Application startup flow

| Step | What happens                                                                               | Important file     |
|------|--------------------------------------------------------------------------------------------|--------------------|
| 1    | XAMPP starts Apache and MySQL. The database must be available before the backend can work. | XAMPP / phpMyAdmin |

|   |                                                                                                                                  |                                                |
|---|----------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------|
| 2 | Backend starts the Express application and connects to MySQL using environment variables.                                        | Backend/server.js and Backend/config/db.js     |
| 3 | Backend registers all route groups such as /auth, /customers, /vehicles, /appointments, /parts, /invoices, and dashboard routes. | Backend/server.js                              |
| 4 | Frontend starts with Vite and renders React pages using BrowserRouter.                                                           | Frontend/src/main.jsx and Frontend/src/App.jsx |
| 5 | When the user logs in, the token, role, userId, first name, and last name are stored in localStorage.                            | Frontend/src/pages/Login.jsx                   |
| 6 | ProtectedRoute checks localStorage before allowing admin, staff, or user pages.                                                  | Frontend/src/routes/ProtectedRoute.jsx         |

## 5. Backend structure

The backend uses Express. In server.js every route file is connected to a base path. For example customer.routes.js is mounted at /customers, vehicle.routes.js is mounted at /vehicles, and invoice.routes.js is mounted at /invoices. That means a route defined as router.get("/") inside customer.routes.js becomes GET /customers in the API.

| server.js base path               | Route file                             | Main responsibility                                                                                                                    |
|-----------------------------------|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| /auth                             | auth.routes.js                         | Register, login, refresh token, profile test route.                                                                                    |
| /customers                        | customer.routes.js                     | CRUD for customers.                                                                                                                    |
| /vehicles                         | vehicle.routes.js                      | CRUD for vehicles and user car status.                                                                                                 |
| /appointments                     | appointment.routes.js                  | CRUD for appointments.                                                                                                                 |
| /service-types                    | serviceType.routes.js                  | CRUD for service categories/types.                                                                                                     |
| /technicians                      | technician.routes.js                   | CRUD for technicians.                                                                                                                  |
| /service-records                  | serviceRecord.routes.js                | Intended for service records. Important note: this route currently imports servicePart.controller instead of serviceRecord.controller. |
| /parts                            | part.routes.js                         | CRUD for parts inventory.                                                                                                              |
| /service-parts                    | servicePart.routes.js                  | CRUD for connecting service records with parts.                                                                                        |
| /invoices                         | invoice.routes.js                      | CRUD for invoices.                                                                                                                     |
| /service-reminders                | serviceReminder.routes.js              | CRUD for service reminders.                                                                                                            |
| /dashboard                        | dashboard.routes.js                    | Dashboard summary endpoints.                                                                                                           |
| /users                            | users.routes.js                        | CRUD for users.                                                                                                                        |
| /roles                            | roles.routes.js                        | CRUD for roles.                                                                                                                        |
| /user-roles                       | userRoles.routes.js                    | CRUD for assigning roles to users.                                                                                                     |
| /user-claims                      | userClaims.routes.js                   | CRUD for claims/permissions.                                                                                                           |
| /user-tokens                      | userTokens.routes.js                   | CRUD for tokens table.                                                                                                                 |
| /mechanic-applications            | mechanicApplication.routes.js          | Public mechanic application form plus admin/staff management.                                                                          |
| /contact-messages                 | contactMessage.routes.js               | Public contact form plus admin/staff management.                                                                                       |
| /emergency-pickups                | emergencyPickup.routes.js              | Public emergency pickup form plus admin/staff management.                                                                              |
| /servicing-requests               | servicingRequest.routes.js             | Public/user servicing request form plus admin/staff management.                                                                        |
| /pre-purchase-inspection-requests | prePurchaseInspectionRequest.routes.js | Public/user inspection request form plus admin/staff management.                                                                       |
| /roadside-subscriptions           | roadsideSubscription.routes.js         | Roadside subscription requests, approve/deny flow.                                                                                     |

## 6. Authentication and authorization

Login starts in Login.jsx. The frontend sends email and password to POST /auth/login. The backend checks the Users table, joins through UserRoles and Roles to get role\_name, compares the password with bcrypt, and then returns a JWT token, refreshToken, and user information. The frontend saves these in localStorage.

Frontend Login.jsx:

```
API.post("/auth/login", { email, password })
localStorage.setItem("token", response.data.token)
localStorage.setItem("role", response.data.user.role)
localStorage.setItem("userId", response.data.user.id)
```

Every protected backend route uses authMiddleware. The frontend Axios instance automatically adds the token to the Authorization header as Bearer <token>. The backend middleware verifies it with JWT\_SECRET. If the token is missing, invalid, or expired, the backend returns 401.

Frontend api.js request flow:

localStorage token -> Authorization: Bearer token -> backend authMiddleware -> jwt.verify -> controller function

Role protection also happens on the frontend. ProtectedRoute receives allowedRoles such as Admin, Staff, User, or Premium User. If the saved role is not allowed, the user is redirected to /profile or /dashboard. The Sidebar also checks the saved role to show only the correct links.

## 7. Frontend routing and page groups

| Page group                     | Frontend routes                                                                                                                                                                                            | Who uses it                                   |
|--------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| Public landing pages           | /, /repairs, /diagnostics, /servicing, /tyres, /tuning, /pre-purchase-inspection, /new-cars, /ev-cars, /suv-cars, /vans, /hybrids                                                                          | Visitors and all users.                       |
| Auth pages                     | /login, /register                                                                                                                                                                                          | Users who need an account or need to sign in. |
| Admin/Staff dashboard          | /dashboard, /customers, /vehicles, /service-types, /technicians, /appointments, /service-records, /parts, /service-parts, /invoices, /service-reminders                                                    | Admin and Staff.                              |
| Admin-only management          | /users, /roles, /user-roles, /user-claims, /user-tokens, /admin-profile                                                                                                                                    | Admin only.                                   |
| Public request pages           | /contact, /mechanic-application, /emergency-pickup, /servicing-requests, /pre-purchase-inspection                                                                                                          | Visitors can submit forms.                    |
| Admin/Staff request management | /dashboard/contact-messages, /dashboard/emergency-pickups, /dashboard/mechanic-applications, /dashboard/servicing-requests, /dashboard/pre-purchase-inspection-requests, /dashboard/roadside-subscriptions | Admin and Staff manage submitted requests.    |
| User self-service              | /profile, /car-status, /my-servicing-requests, /my-pre-purchase-inspection-requests, /roadside-subscription                                                                                                | Normal users and premium users.               |

## 8. CRUD pattern used by the project

Most CRUD pages use the same structure. When the page loads, useEffect calls fetchData. fetchData sends API.get to the backend and saves the result in React state. A form is used for create and update. If no editing id exists, the page calls POST. If an editing id exists, the page calls PUT. Delete buttons call DELETE and then reload the data.

Typical CRUD connection:

React page state -> form input -> API.post or API.put -> Express route -> controller -> SQL INSERT/UPDATE -> MySQL table -> JSON response -> refresh frontend table

| CRUD action | Frontend action                        | HTTP method | Backend route pattern | Controller action | Database operation                   |
|-------------|----------------------------------------|-------------|-----------------------|-------------------|--------------------------------------|
| Read all    | Page loads data into table             | GET         | /entity               | getEntityPlural   | SELECT * FROM table                  |
| Read one    | Edit/details can load a single row     | GET         | /entity/:id           | getEntityById     | SELECT * FROM table WHERE id = ?     |
| Create      | Submit empty/new form                  | POST        | /entity               | createEntity      | INSERT INTO table (...) VALUES (...) |
| Update      | Submit form while editing existing row | PUT         | /entity/:id           | updateEntity      | UPDATE table SET ... WHERE id = ?    |
| Delete      | Click delete button                    | DELETE      | /entity/:id           | deleteEntity      | DELETE FROM table WHERE id = ?       |

## 9. CRUD connection map

| Feature       | Frontend page    | API base       | Route file            | Controller                | DB table     | Operations                                                       |
|---------------|------------------|----------------|-----------------------|---------------------------|--------------|------------------------------------------------------------------|
| Customers     | Customers.jsx    | /customers     | customer.routes.js    | customer.controller.js    | Customers    | GET, GET/:id, POST, PUT/:id, DELETE/:id                          |
| Vehicles      | Vehicles.jsx     | /vehicles      | vehicle.routes.js     | vehicle.controller.js     | Vehicles     | GET, GET/:id, POST, PUT/:id, DELETE/:id, plus /my-status/:userId |
| Appointments  | Appointments.jsx | /appointments  | appointment.routes.js | appointment.controller.js | Appointments | GET, GET/:id, POST, PUT/:id, DELETE/:id                          |
| Service Types | ServiceTypes.jsx | /service-types | serviceType.routes.js | serviceType.controller.js | ServiceTypes | GET, GET/:id, POST, PUT/:id, DELETE/:id                          |
| Technicians   | Technicians.jsx  | /technicians   | technician.routes.js  | technician.controller.js  | Technicians  | GET, GET/:id, POST,                                              |

|                        |                                                                           |                                   |                                        |                                            |                                  |                                                                                            |
|------------------------|---------------------------------------------------------------------------|-----------------------------------|----------------------------------------|--------------------------------------------|----------------------------------|--------------------------------------------------------------------------------------------|
| Service Records        | ServiceRecords.jsx                                                        | /service-records                  | serviceRecord.routes.js                | serviceRecord.controller.js intended       | ServiceRecords                   | PUT/:id, DELETE/:id<br>intended CRUD, but route currently points to servicePart.controller |
| Parts                  | Parts.jsx                                                                 | /parts                            | part.routes.js                         | part.controller.js                         | Parts                            | GET, GET/:id, POST, PUT/:id, DELETE/:id                                                    |
| Service Parts          | ServiceParts.jsx                                                          | /service-parts                    | servicePart.routes.js                  | servicePart.controller.js                  | ServiceParts                     | GET, GET/:id, POST, PUT/:id, DELETE/:id                                                    |
| Invoices               | Invoices.jsx                                                              | /invoices                         | invoice.routes.js                      | invoice.controller.js                      | Invoices                         | GET, GET/:id, POST, PUT/:id, DELETE/:id                                                    |
| Service Reminders      | ServiceReminders.jsx                                                      | /service-reminders                | serviceReminder.routes.js              | serviceReminder.controller.js              | ServiceReminders                 | GET, GET/:id, POST, PUT/:id, DELETE/:id                                                    |
| Users                  | Users.jsx                                                                 | /users                            | users.routes.js                        | users.controller.js                        | Users                            | GET, GET/:id, POST, PUT/:id, DELETE/:id                                                    |
| Roles                  | Roles.jsx                                                                 | /roles                            | roles.routes.js                        | roles.controller.js                        | Roles                            | GET, GET/:id, POST, PUT/:id, DELETE/:id                                                    |
| User Roles             | UserRoles.jsx                                                             | /user-roles                       | userRoles.routes.js                    | userRoles.controller.js                    | UserRoles                        | GET, GET/:id, POST, PUT/:id, DELETE/:id                                                    |
| User Claims            | UserClaims.jsx                                                            | /user-claims                      | userClaims.routes.js                   | userClaims.controller.js                   | UserClaims                       | GET, GET/:id, POST, PUT/:id, DELETE/:id                                                    |
| User Tokens            | UserTokens.jsx                                                            | /user-tokens                      | userTokens.routes.js                   | userTokens.controller.js                   | UserTokens                       | GET, GET/:id, POST, PUT/:id, DELETE/:id                                                    |
| Contact Messages       | ContactMessages.jsx                                                       | /contact-messages                 | contactMessage.routes.js               | contactMessage.controller.js               | contact_messages                 | GET protected, POST public, PUT status, DELETE protected                                   |
| Emergency Pickups      | EmergencyPickups.jsx and EmergencyPickup.jsx                              | /emergency-pickups                | emergencyPickup.routes.js              | emergencyPickup.controller.js              | emergency_pickups                | GET protected, POST public, PUT status, DELETE protected                                   |
| Mechanic Applications  | MechanicApplications.jsx and MechanicApplication.jsx                      | /mechanic-applications            | mechanicApplication.routes.js          | mechanicApplication.controller.js          | mechanic_applications            | GET protected, POST public with CV upload, PUT status, DELETE protected                    |
| Servicing Requests     | ServicingRequests.jsx and MyServicingRequests.jsx                         | /servicing-requests               | servicingRequest.routes.js             | servicingRequest.controller.js             | servicing_requests               | GET all protected, GET /user/:userId protected, POST public, PUT, DELETE                   |
| Inspection Requests    | PrePurchaseInspectionRequests.jsx and MyPrePurchaseInspectionRequests.jsx | /pre-purchase-inspection-requests | prePurchaseInspectionRequest.routes.js | prePurchaseInspectionRequest.controller.js | pre_purchase_inspection_requests | GET all protected, GET /user/:userId protected, POST public, PUT, DELETE                   |
| Roadside Subscriptions | RoadsideSubscription.jsx and RoadsideSubscriptions.jsx                    | /roadside-subscriptions           | roadsideSubscription.routes.js         | roadsideSubscription.controller.js         | roadside_subscription_requests   | GET all protected, GET /user/:userId, POST protected, approve, deny, DELETE                |

## 10. Example CRUD flow: Customers

The Customers module is the clearest example of how CRUD is connected. Customers.jsx calls GET /customers to load rows. customer.routes.js protects the route with authMiddleware and calls getCustomers. customer.controller.js executes SELECT \* FROM Customers and returns JSON to the frontend table.

| Operation | Frontend                                      | Backend route         | Controller SQL idea                                                         |
|-----------|-----------------------------------------------|-----------------------|-----------------------------------------------------------------------------|
| Read all  | API.get("/customers")                         | GET /customers        | SELECT * FROM Customers                                                     |
| Read one  | Used when an id is needed                     | GET /customers/:id    | SELECT * FROM Customers WHERE id = ?                                        |
| Create    | API.post("/customers", formData)              | POST /customers       | INSERT INTO Customers (emri, mbiemri, telefoni, email, adresa) VALUES (...) |
| Update    | API.put('/customers/\${editingId}', formData) | PUT /customers/:id    | UPDATE Customers SET ... WHERE id = ?                                       |
| Delete    | API.delete('/customers/\${id}')               | DELETE /customers/:id | DELETE FROM Customers WHERE id = ?                                          |

## 11. Example connected flow: user car status

The car status page is a special read-only flow for normal users. CarStatus.jsx reads userId from localStorage and calls GET /vehicles/my-status/:userId. vehicle.controller.js joins Vehicles with Customers and ServiceRecords. This lets a user see their vehicles and the latest related service status.

CarStatus.jsx -> API.get('/vehicles/my-status/\${userId}')

vehicle.routes.js -> router.get("/my-status/:userId", authMiddleware, getMyVehicles)

vehicle.controller.js -> Vehicles JOIN Customers LEFT JOIN ServiceRecords WHERE Customers.user\_id = ?

## 12. Dashboard connection

Dashboard.jsx calls several dashboard endpoints. These endpoints are read-only summary endpoints for admin/staff. They are useful for quick statistics such as appointments, revenue, stock, customers, vehicles, invoices, and low stock parts.

| Frontend API call                        | Backend endpoint                  | Purpose                          |
|------------------------------------------|-----------------------------------|----------------------------------|
| API.get("/dashboard/today-appointments") | GET /dashboard/today-appointments | Shows appointments for today.    |
| API.get("/dashboard/revenue")            | GET /dashboard/revenue            | Shows invoice/revenue total.     |
| API.get("/dashboard/parts-stock")        | GET /dashboard/parts-stock        | Shows total parts stock.         |
| API.get("/dashboard/total-customers")    | GET /dashboard/total-customers    | Counts customers.                |
| API.get("/dashboard/total-vehicles")     | GET /dashboard/total-vehicles     | Counts vehicles.                 |
| API.get("/dashboard/total-appointments") | GET /dashboard/total-appointments | Counts appointments.             |
| API.get("/dashboard/total-invoices")     | GET /dashboard/total-invoices     | Counts invoices.                 |
| API.get("/dashboard/low-stock-parts")    | GET /dashboard/low-stock-parts    | Lists parts that need attention. |

## 13. Public request forms and staff management

Several pages are designed for visitors or users to submit requests. The POST route is public in most of these modules, while GET/PUT/DELETE are protected for admin/staff. This is a good design because visitors can send requests without being staff, while staff can manage the requests in the dashboard.

| Public form                           | POST endpoint                            | Admin/Staff management page                                               | Common status flow                                                      |
|---------------------------------------|------------------------------------------|---------------------------------------------------------------------------|-------------------------------------------------------------------------|
| Contact.jsx                           | POST /contact-messages                   | ContactMessages.jsx                                                       | New message -> status updated by staff -> delete when no longer needed. |
| EmergencyPickup.jsx                   | POST /emergency-pickups                  | EmergencyPickups.jsx                                                      | Request submitted -> staff changes status -> delete if needed.          |
| MechanicApplication.jsx               | POST /mechanic-applications with cv_file | MechanicApplications.jsx                                                  | Applicant submits CV -> staff changes status -> delete if needed.       |
| Servicing.jsx / ServicingRequests.jsx | POST /servicing-requests                 | ServicingRequests.jsx and MyServicingRequests.jsx                         | User request -> staff status update -> user can view own requests.      |
| PrePurchaseInspection.jsx             | POST /pre-purchase-inspection-requests   | PrePurchaseInspectionRequests.jsx and MyPrePurchaseInspectionRequests.jsx | User request -> staff status update -> user can view own requests.      |
| RoadsideSubscription.jsx              | POST /roadside-subscriptions             | RoadsideSubscriptions.jsx                                                 | User subscribes -> staff approve or deny.                               |

## 14. Database relationship explanation

The database is organized around users, customers, vehicles, and service work. Users are accounts for login. Customers represent service customers. Vehicles belong to customers. Appointments and service records connect customers/vehicles to service work. Parts and service parts connect inventory to services. Invoices connect payments/costs to service work. Roles and user roles control access.

| Relationship                                                | Meaning in the project                                                                                                                         |
|-------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Users -> UserRoles -> Roles                                 | A user can receive a role such as Admin, Staff, User, or Premium User. Login uses this to decide where to redirect and what pages are allowed. |
| Users -> Customers                                          | A customer can be linked to a user account. This allows the user to view car status and personal requests.                                     |
| Customers -> Vehicles                                       | Each vehicle belongs to a customer. Vehicle CRUD requires customer_id.                                                                         |
| Vehicles -> ServiceRecords                                  | Service records describe work/status for a vehicle. The car status page uses this connection.                                                  |
| ServiceRecords -> ServiceParts -> Parts                     | Service parts connect used parts to a service record and reduce/track inventory.                                                               |
| Appointments -> Customers/Vehicles/Technicians/ServiceTypes | Appointments schedule work for a customer/vehicle and can be assigned service details and technicians.                                         |
| Invoices -> ServiceRecords/Customers                        | Invoices represent charges and payment information for service work.                                                                           |

|                          |                                                                                                              |
|--------------------------|--------------------------------------------------------------------------------------------------------------|
| Requests tables -> Users | Servicing, inspection, and roadside requests can connect to user_id so users can view their own submissions. |
|--------------------------|--------------------------------------------------------------------------------------------------------------|

## 15. Security and access control summary

| Security feature            | How it works                                                                                    | Files involved                                                                     |
|-----------------------------|-------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Password hashing            | Passwords are hashed with bcrypt before saving.                                                 | auth.controller.js and users.controller.js                                         |
| JWT token                   | Login returns a signed token with user id and email.                                            | auth.controller.js                                                                 |
| Protected backend APIs      | Most routes require authMiddleware before controller functions run.                             | Backend/routes/*.routes.js and auth.middleware.js                                  |
| Protected frontend pages    | ProtectedRoute checks token and role from localStorage.                                         | ProtectedRoute.jsx and App.jsx                                                     |
| Role-based sidebar          | Sidebar shows different navigation links depending on saved role.                               | Sidebar.jsx                                                                        |
| Public-only form submission | Some POST endpoints are public so visitors can submit contact, emergency, or application forms. | contactMessage.routes.js, emergencyPickup.routes.js, mechanicApplication.routes.js |

## 16. Important observations from the code review

The project is generally connected well: frontend pages map to backend routes, backend routes map to controllers, controllers run SQL queries, and the database tables match the main modules. However, while checking the files carefully, I found a few important points that should be fixed or at least explained before final submission.

| Observation                                                                                                                                                    | Why it matters                                                                                                                      | Recommended fix                                                                                                                                                                                                                                                                           |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Many CRUD pages have delete calls like <code>API.delete(`/customers/\${id}`)</code> even when the page is Appointments, Parts, Invoices, Roles, Vehicles, etc. | This means delete buttons on those pages may delete customers or fail instead of deleting the correct entity.                       | Change each delete call to its correct endpoint, for example <code>/vehicles/\${id}</code> , <code>/parts/\${id}</code> , <code>/appointments/\${id}</code> , <code>/invoices/\${id}</code> , <code>/roles/\${id}</code> .                                                                |
| <code>serviceRecord.routes.js</code> imports functions from <code>servicePart.controller.js</code> .                                                           | The <code>/service-records</code> backend route may actually operate on service parts instead of service records.                   | Change <code>serviceRecord.routes.js</code> to import from <code>serviceRecord.controller.js</code> and use <code>getServiceRecords</code> , <code>getServiceRecordById</code> , <code>createServiceRecord</code> , <code>updateServiceRecord</code> , <code>deleteServiceRecord</code> . |
| <code>Register.jsx</code> posts to <code>/users</code> , while <code>auth.routes.js</code> also has <code>/auth/register</code> .                              | Registration may work through <code>users.controller.js</code> , but the dedicated auth register route is not used by the frontend. | Use one consistent registration flow. Usually <code>POST /auth/register</code> is cleaner because it is part of authentication.                                                                                                                                                           |
| Refresh token is returned from <code>/auth/login</code> , but the visible <code>auth.controller.js</code> does not store it in <code>UserTokens</code> .       | The <code>UserTokens</code> CRUD exists, but login refresh tokens are not visibly connected to the table.                           | Either store refresh tokens in <code>UserTokens</code> or explain that <code>UserTokens</code> is managed manually for project CRUD requirements.                                                                                                                                         |
| Backend <code>package.json</code> does not define a start script.                                                                                              | The user must run <code>node server.js</code> manually instead of <code>npm start</code> .                                          | Add "start": "node server.js" and optionally "dev": "nodemon server.js".                                                                                                                                                                                                                  |

## 17. How to explain the CRUD connection in presentation

A simple explanation for presentation:

When I open a CRUD page such as Customers, the React page sends a GET request to the backend. The backend route checks the JWT token using `authMiddleware` and then calls the controller. The controller runs an SQL query using the `mysql2` connection. The result comes back as JSON and is displayed in the React table. When I add, edit, or delete data, the same path is used but with POST, PUT, or DELETE instead of GET.

For example, Customers CRUD is connected like this: `Customers.jsx` -> `API.get/post/put/delete` -> `/customers` route -> `customer.controller.js` -> Customers table. The same pattern is repeated for vehicles, appointments, parts, invoices, technicians, service types, users, roles, and the other modules.

## 18. Suggested final testing checklist

| Test                         | Expected result                         |
|------------------------------|-----------------------------------------|
| Start XAMPP MySQL and Apache | MySQL and phpMyAdmin open successfully. |

|                                                           |                                                                                                                           |
|-----------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Start backend with node server.js                         | Console shows Server running on port 5000 and Connected to MySQL database.                                                |
| Start frontend with npm run dev                           | Vite opens the React app.                                                                                                 |
| Login as Admin                                            | Redirects to dashboard and sidebar shows admin links.                                                                     |
| Login as Staff                                            | Redirects to dashboard and sidebar hides admin-only user/role/token pages.                                                |
| Login as User/Premium User                                | Redirects to profile and user pages only.                                                                                 |
| Test Customers CRUD                                       | Can add, view, edit, and delete customers.                                                                                |
| Test Vehicles, Appointments, Parts, Invoices, Technicians | Can add, view, edit, and delete after fixing wrong delete endpoints.                                                      |
| Submit public forms                                       | Contact, emergency pickup, mechanic application, servicing, and inspection requests appear in dashboard management pages. |
| Test car status                                           | User sees their vehicles and latest service status through /vehicles/my-status/:userId.                                   |
| Test dashboard                                            | Cards display totals and low stock parts correctly.                                                                       |

## 19. Conclusion

The Vehicle Service Management System is structured like a typical full stack CRUD project. The frontend is organized into public pages, protected admin/staff pages, and protected user pages. The backend is organized into route files and controller files, with MySQL queries inside the controllers. The connection between CRUD pages and database tables is mostly clear and consistent. Before final submission, the most important fixes are the wrong delete endpoints in several frontend pages and the service records route importing the wrong controller.

After those fixes, the project explanation is strong: React handles UI, Axios sends requests, Express routes receive them, controllers perform SQL operations, MySQL stores the data, JWT protects private areas, and roles decide what each user can access.